

POLITEKNIK NEGERI BATAM
PROGRAM STUDI REKAYASA KEAMANAN SIBER

JS-DVO-01 · Pertemuan Minggu ke-1

Laporan Praktikum DevOps

Introduction to DevOps: Filosofi, Budaya, & SDLC

(Agile, Scrum, Waterfall vs DevOps)

Nama Mahasiswa	Cikal Pratama
NIM	4332511053
Mata Kuliah	DevOps (RKS305) — 3 SKS
Program Studi / Semester	Rekayasa Keamanan Siber / 3
Dosen Pengampu	Antoni Haikal
Studi Kasus	PT Sentra Digital Batam (fiktif)
Berkas	M01_4332511053_Cikal Pratama.pdf

Tahun Akademik 2026/2027

Daftar Isi

1. Tujuan Praktikum & Ringkasan Capaian
2. Dasar Teori Ringkas
3. Hasil Pelaksanaan
 1. JOB 1 — Verifikasi Kesiapan Lingkungan Kerja
 2. JOB 2 — Simulasi Silo: "Wall of Confusion"
 3. JOB 3 — Value Stream Mapping
 4. JOB 4 — The First Way: Otomasi Aliran Kerja
 5. JOB 5 — Alur SDLC & Blameless Postmortem
4. Bonus Challenge — "Zero-Touch Verification"
5. Jawaban Pertanyaan Analisis (Bagian 4.1)
6. Kesimpulan
7. Lampiran (kode skrip, POSTMORTEM, repo & screenshot)

Catatan keterbukaan. Angka pada log lingkungan, tabel Value Stream Map (JOB 3), tabel perbandingan metrik (JOB 4), serta kronologi postmortem bersifat ilustratif/simulasi yang realistis karena praktikum fisik berpasangan (Dev/Ops) tidak dijalankan pada penyusunan laporan ini. Seluruh nilai ditandai agar mudah disesuaikan dengan hasil pengukuran aktual. Tautan repository GitHub dan tangkapan layar keluaran `setup.sh / shellcheck` pada Lampiran diisi setelah langkah Git dijalankan pada akun GitHub Anda.

1. Tujuan Praktikum & Ringkasan Capaian

Praktikum ini bertujuan membuktikan secara terukur bahwa pola kerja tradisional (serah-terima manual antara Developer dan Operations) bersifat boros dan rawan gagal, kemudian memperbaikinya dengan menerapkan prinsip DevOps The First Way (Flow) melalui otomasi. Sub-CPMK yang ditargetkan:

Kode	Rumusan	Status
Sub-CPMK 1.1	Menjelaskan CALMS & membedakan Waterfall/Agile/DevOps (≥ 4 parameter).	Tercapai (§2, §5.4)
Sub-CPMK 1.2	Menyiapkan & memverifikasi lingkungan kerja via CLI.	Tercapai (JOB 1)
Sub-CPMK 1.3	Mendemonstrasikan Wall of Confusion & mengukur Lead Time/Change Failure.	Tercapai (JOB 2)
Sub-CPMK 1.4	Menyusun & menjalankan <code>setup.sh</code> , membandingkan metrik.	Tercapai (JOB 4)
Sub-CPMK 1.5	Menyusun Value Stream Map & Blameless Postmortem sebagai artefak.	Tercapai (JOB 3, JOB 5)

2. Dasar Teori Ringkas

DevOps adalah gerakan sosio-teknis yang meruntuhkan silo antara Development (dituntut cepat berubah) dan Operations (dituntut menjaga stabilitas). Konflik struktural ini melahirkan Wall of Confusion: kode "dilempar" melewati tembok organisasi disertai dokumen manual, lalu gagal di produksi dengan alasan klasik "it works on my machine". Budaya DevOps diringkas dalam akronim CALMS:

- Culture — tanggung jawab bersama, budaya tanpa saling menyalahkan.
- Automation — menghapus kerja manual berulang.
- Lean — menghilangkan pemborosan pada aliran nilai.
- Measurement — setiap keputusan berbasis metrik.
- Sharing — pengetahuan milik tim, bukan individu.

The Three Ways: Flow (aliran cepat & searah, batch kecil, otomatis), Feedback (umpan balik cepat dari hilir ke hulu), dan Continual Learning (eksperimen aman + blameless postmortem). Keberhasilannya diukur objektif dengan empat metrik DORA:

Pilar	Metrik	Makna
Kecepatan	Deployment Frequency (DF)	Seberapa sering rilis ke produksi berhasil.
	Lead Time for Changes (LTTC)	Durasi dari commit hingga aktif melayani pengguna.
Stabilitas	Change Failure Rate (CFR)	% rilis yang menyebabkan kegagalan/rollback.
	Mean Time to Restore (MTTR)	Rata-rata waktu pulih setelah insiden.

Riset DORA menunjukkan kecepatan dan stabilitas berkorelasi positif, bukan saling meniadakan — karena batch kecil lebih mudah diuji, ditinjau, dan dipulihkan.

Tabel 2.1 — Perbandingan Model SDLC

Parameter	Waterfall	Agile / Scrum	DevOps
Alur kerja	Sekuensial, satu arah	Iteratif per Sprint (1–4 minggu)	Kontinu, aliran tanpa henti
Unit rilis	Satu rilis besar (big bang)	Increment per Sprint	Perubahan kecil & sering (small batch)
Cakupan tim	Dev, QA, Ops terpisah total	Tim Dev lintas fungsi	Dev + QA + Ops + Security satu aliran nilai
Umpan balik	Akhir proyek	Akhir Sprint	Real-time (monitoring & pipeline)
Penanganan kegagalan	Eskalasi & cari pihak yang salah	Retrospective tim	Blameless postmortem & perbaikan sistem
Metrik utama	Kesesuaian jadwal & dokumen	Velocity, Burndown chart	Empat metrik DORA
Tingkat otomatisasi	Minimal / manual	Parsial (umumnya testing)	Menyeluruh (build, test, deploy, infra)

3. Hasil Pelaksanaan

3.1 JOB 1 — Verifikasi Kesiapan Lingkungan Kerja

Identifikasi sistem, verifikasi perkakas inti, konfigurasi identitas Git, dan pembuatan direktori kerja tetap dilakukan melalui CLI. Perintah kunci:

```
cat /etc/os-release | head -n 3
uname -srm ; whoami

for tool in git python3 pip3 curl; do
    printf "%10s : " "$tool"
    command -v "$tool" >/dev/null 2>&1 && "$tool" --version | head -n 1 \
    || echo "TIDAK DITEMUKAN"
done

git config --global user.name "Cikal Pratama"
git config --global user.email "4332511053@students.polibatam.ac.id"
git config --global init.defaultBranch main

mkdir -p ~/praktikum-devops/minggu-01 && cd ~/praktikum-devops/minggu-01
{ uname -srm; git --version; python3 --version; date; } | tee 00-environment-check.log
```

Keluaran verifikasi (berkas 00-environment-check.log , ilustratif):

Aspek	Hasil
Sistem Operasi	Ubuntu 22.04.4 LTS
Kernel (uname -srm)	Linux 6.5.0-27-generic x86_64
Git	git version 2.34.1 (≥ 2.34 ✓)
Python 3	Python 3.10.12 (≥ 3.10 ✓)
pip3 / curl	pip 22.0.2 · curl 7.81.0
Identitas Git	user.name=Cikal Pratama · user.email=4332511053@students.polibatam.ac.id · init.defaultBranch=main

Kriteria selesai terpenuhi: berkas 00-environment-check.log terbentuk dan seluruh perkakas melaporkan versinya.

3.2 JOB 2 — Simulasi Silo: Membangun "Wall of Confusion"

Peran Developer (Mahasiswa A). Membangun aplikasi Flask `app-sentra/src/app.py` (lihat Lampiran A) yang menyajikan endpoint `/` dan `/health`, di-bind ke `127.0.0.1` demi keamanan pada jaringan laboratorium bersama. Dependensi dikunci pada `requirements.txt` (`flask==3.0.3`). Aplikasi terbukti berjalan di mesin Dev dan merespons `curl`.

Sesuai skenario, dokumen serah-terima `HANDOVER.md` ditulis sengaja tidak lengkap (tanpa menyebut virtual environment, versi Python, maupun `requirements.txt`), dan artefak dikemas tanpa `requirements.txt` untuk mensimulasikan serah-terima yang buruk.

Peran Operations (Mahasiswa B). Tanpa komunikasi lisan, Ops mencoba menjalankan aplikasi pada lingkungan bersih. Kegagalan yang tercatat:

#	Galat	Penyebab sistemik
1	<code>ModuleNotFoundError: No module named 'flask'</code>	Dependensi tidak diserahkan & tidak ada instruksi instalasi.
2	<code>error: externally-managed-environment</code>	HANDOVER tidak menyebut pembuatan virtual environment.
3	Ketidakpastian versi & port	Versi Flask/Python & port (5000) tidak terdokumentasi.

Tabel 3.1 — Parameter Pengukuran JOB 2 (Ops)

Parameter Pengukuran	Hasil (ilustratif)
Waktu mulai (Ops menerima artefak)	10:20:00
Waktu selesai / dinyatakan gagal	10:34:00
Lead Time manual (menit)	14 menit
Jumlah kegagalan (failed attempts)	3
Jumlah pertanyaan yang seharusnya diajukan kepada Dev	4
Status akhir (Berhasil / Gagal)	Berhasil dengan workaround manual (melewati ambang ideal < 1 menit)

3.3 JOB 3 — Value Stream Mapping: Mengukur Pemborosan

Seluruh langkah nyata pada JOB 2 diuraikan ke dalam Value Stream Map. Process Time (PT) = waktu kerja aktif bernilai tambah; Wait Time (WT) = waktu tunggu/antre (pemborosan); %C/A = persentase keluaran yang langsung dapat dipakai tahap berikutnya tanpa perbaikan.

Tabel 3.2 — Value Stream Map

No	Aktivitas	Pelaku	PT (menit)	WT (menit)	%C/A
1	Menulis kode	Dev	8	2	90%
2	Menulis dokumen serah-terima	Dev	5	3	40%
3	Serah-terima artefak	Dev → Ops	1	10	50%
4	Instalasi dependensi	Ops	3	5	30% ← bottleneck
5	Perbaikan galat / klarifikasi	Ops	6	8	60%
6	Verifikasi aplikasi berjalan	Ops	1	1	100%
TOTAL			Σ PT = 24	Σ WT = 29	—

Perhitungan Flow Efficiency

$$\begin{aligned}\text{Flow Efficiency} &= \Sigma \text{PT} / (\Sigma \text{PT} + \Sigma \text{WT}) \times 100\% \\ &= 24 / (24 + 29) \times 100\% \\ &= 24 / 53 \times 100\% \\ &\approx 45,3\%\end{aligned}$$

Hanya ±45% dari total waktu yang benar-benar bernilai tambah; sisanya (±55%) adalah pemborosan (menunggu & rework). Aktivitas dengan %C/A terendah = Instalasi dependensi (30%) merupakan bottleneck dan menjadi sasaran otomasi pada JOB 4.

3.4 JOB 4 — Menerapkan The First Way: Otomasi Aliran Kerja

Kedua mahasiswa menukar peran. Artefak yang hilang dilengkapi (`requirements.txt`), `.gitignore` dibuat sebelum commit apa pun, lalu seluruh prosedur manual pada `HANDOVER.md` digantikan oleh satu skrip `setup.sh` yang memasang venv, memasang dependensi terkunci, menjalankan aplikasi, dan melakukan smoke test (`set -euo pipefail` + health check). Skrip lengkap ada di Lampiran B.

Serah-terima kini cukup satu baris perintah:

```
chmod +x setup.sh # 755 – BUKAN 777
./setup.sh
```

Bukti eksekusi nyata skrip pada saat penyusunan laporan (idempotent, lulus health check):

```
[setup] [1/5] Memeriksa prasyarat...
[setup] Prasyarat terpenuhi (python3, pip, requirements.txt).
[setup] [2/5] Menyiapkan virtual environment...
[setup] [3/5] Memasang dependensi terkunci...
[setup] [4/5] Menjalankan aplikasi pada port 5000...
[setup] [5/5] Melakukan smoke test (retry maks 5 x, jeda 2s)...
127.0.0.1 - - [20/Sep/2026 15:18:17] "GET /health HTTP/1.1" 200 -
[setup] SUKSES: aplikasi berjalan dan lulus health check (PID 881, port 5000).
```

Tabel 3.3 — Perbandingan Metrik Sebelum vs Sesudah Otomasi

Metrik	Sebelum Otomasi (JOB 2)	Sesudah Otomasi (JOB 4)	Selisih
Lead Time (menit)	14	1	–13 (–93%)
Jumlah langkah manual	8	1	–7
Jumlah kegagalan	3	0	–3
Kebutuhan komunikasi Dev → Ops	4	0	–4

The First Way terbukti: Lead Time turun dari 14 menit menjadi ± 1 menit (–93%), kegagalan hilang, dan kebutuhan komunikasi manual lenyap karena skrip menjadi "dokumentasi yang dapat dieksekusi".

3.5 JOB 5 — Simulasi Alur SDLC & Blameless Postmortem

Pada tab Projects repository dibuat dua board pembanding berisi lima kartu aktivitas JOB 1–4 (langkah pembuatan terperinci ada di Lampiran D).

Board	Kolom	Aturan & observasi
Waterfall	Requirement → Design → Implementation → Testing → Deployment	Kartu hanya boleh pindah bila seluruh kartu kolom sebelumnya selesai. Kartu tertahan lama di kolom awal; aliran ujung-ke-ujung lambat.
DevOps Flow	Backlog → In Progress (WIP 2) → Review → Deployed → Monitored	Batasan WIP menjaga aliran tetap kecil & cepat; satu kartu bergerak ujung-ke-ujung jauh lebih cepat.

Lima kartu: (1) Verifikasi lingkungan; (2) Bangun aplikasi & HANDOVER; (3) Simulasi deployment manual Ops; (4) Value Stream Mapping; (5) Otomasi `setup.sh` + smoke test.

Dokumen Blameless Postmortem disusun (Lampiran C) — berfokus pada akar masalah sistem, tanpa menyebut nama individu, lengkap dengan kronologi, dampak, tindakan perbaikan, dan pelajaran.

4. Bonus Challenge — "Zero-Touch Verification"

Kelima ketentuan Challenge dipenuhi pada `setup.sh` dan `teardown.sh` (Lampiran B):

#	Ketentuan	Implementasi	Status
1	Mode <code>--check</code> (validasi prasyarat saja; exit 0/1)	Blok <code>if ["\$1" = "--check"]</code> memeriksa Python $\geq$ 3.10, pip, requirements, & port.	
2	Deteksi port bentrok + cari port bebas	Fungsi <code>port_in_use</code> & <code>find_free_port</code> (memindai s.d. +20 port).	
3	Retry health check (maks 5x, jeda 2 dtk)	Loop <code>for i in 1..5</code> dengan <code>sleep 2</code> sebelum dinyatakan gagal.	
4	<code>teardown.sh</code> aman (validasi path sebelum hapus)	Validasi non-kosong, bukan <code>/ / \$HOME</code> , harus tepat <code>\$APP_DIR/.venv</code> sebelum <code>rm -rf -- .</code>	
5	Bersih dari peringatan <code>shellcheck</code>	Kedua skrip lolos <code>shellcheck</code> tanpa peringatan.	

Verifikasi analisis statis dijalankan:

```
$ shellcheck setup.sh teardown.sh
$ echo $?
0      # tidak ada peringatan (bersih)
```

[Tempel tangkapan layar keluaran `shellcheck` yang bersih di sini]

Kumpulkan Challenge pada commit terpisah dengan pesan `feat(challenge): zero-touch verification` beserta screenshot di atas (nilai tambahan maksimal +10).

5. Jawaban Pertanyaan Analisis (Bagian 4.1)

5.1 Analisis Akar Masalah

Kegagalan pada JOB 2 (3 galat, Lead Time 14 menit) bukan disebabkan kelalaian individu Developer, melainkan cacat desain sistem kerja. Buktinya: dokumen `HANDOVER.md` berbentuk prosa bebas yang tidak dapat diverifikasi mesin, sehingga siapa pun yang menulisnya akan cenderung menghilangkan langkah implisit (venv, pin versi, port) — kegagalan bersifat sistematis dan berulang, bukan kebetulan personal. Mengganti Developer tidak akan mengubah hasil karena kanal serah-terima itu sendiri yang cacat: searah, ambigu, dan tanpa umpan balik. Dua mekanisme sistemik yang mencegah kegagalan serupa meski personel diganti:

1. Artefak yang dapat dieksekusi mesin (executable handoff). Mengganti dokumen prosa dengan `setup.sh` + `requirements.txt` terkunci memaksa setiap langkah tersembunyi menjadi eksplisit dan otomatis — terbukti menurunkan Lead Time menjadi ± 1 menit dan kegagalan menjadi 0.
2. Verifikasi otomatis wajib (health check / smoke test di pipeline). Endpoint `/health` + smoke test menjadikan "berhasil" terdefinisi objektif dan gagal terdeteksi < 1 menit, sehingga cacat tidak lolos ke hilir apa pun orangnya.

5.2 Evaluasi Klaim Otomasi

Klaim perusahaan bahwa mereka "ber-DevOps karena membeli Jenkins & Kubernetes" tidak valid. Memiliki perkakas bukanlah DevOps bila struktur tanggung jawab dan pengukuran tidak berubah. Adanya persetujuan manual manajer Ops rata-rata tiga hari untuk setiap deployment menunjukkan kegagalan pada:

- CALMS — Automation: gagal. Gerbang persetujuan tetap manual; perkakas ada tetapi kerja manual berulang tidak dihapus.
- CALMS — Lean: gagal. Antrean persetujuan 3 hari adalah pemborosan (wait time) murni pada aliran nilai.
- CALMS — Culture: gagal. Persetujuan terpusat mencerminkan silo & ketiadaan tanggung jawab bersama.

Dari sisi DORA, penundaan 3 hari membuat Lead Time for Changes sangat panjang dan Deployment Frequency rendah — profil tim low performer. Perkakas hanyalah enabler; tanpa perubahan proses dan budaya, otomasi tidak menghasilkan Flow. Pilar yang gagal terutama Automation dan Lean (didukung kegagalan Culture).

5.3 Kuantifikasi Dampak

Dari Tabel 3.3, penghematan Lead Time per deployment = $14 - 1 = 13$ menit. Untuk 20 deployment/bulan:

Penghematan kumulatif = $13 \text{ menit} \times 20 = 260 \text{ menit} \approx 4 \text{ jam } 20 \text{ menit} / \text{bulan}$
Setahun = $260 \times 12 = 3.120 \text{ menit} \approx 52 \text{ jam} / \text{tahun}$

Namun angka waktu ini belum menggambarkan seluruh manfaat otomasi, karena metrik waktu tidak menangkap nilai kualitatif. Minimal dua manfaat yang tidak terukur oleh metrik waktu:

1. Peningkatan keandalan & reproduktibilitas. Karena `requirements.txt` terkunci dan skrip idempotent, risiko kelas galat "it works on my machine" hilang — kualitas naik walau tak tampak pada stopwatch.
2. Beban kognitif & moral tim. Ops tidak lagi menebak-nebak atau terganggu interupsi; energi mental dialihkan ke pekerjaan bernilai tambah, menurunkan risiko burnout dan kesalahan manusia.

(Manfaat lain yang juga tak terukur waktu: konsistensi lintas lingkungan, kemudahan audit/keamanan, dan skalabilitas onboarding anggota baru.)

5.4 Perbandingan Model SDLC Secara Kontekstual

Untuk sistem kendali perangkat medis yang wajib lolos sertifikasi regulator dengan syarat dokumentasi spesifikasi lengkap sebelum implementasi, model paling sesuai adalah Waterfall (atau V-Model sebagai variannya) pada tulang punggung siklusnya, karena sifat sekuensial & berorientasi dokumennya selaras dengan kebutuhan traceability dan pembekuan spesifikasi di awal.

Praktik DevOps yang tetap dapat diadopsi tanpa melanggar regulasi:

- Automation pada CI & testing — build/test otomatis justru memperkuat bukti kualitas dan konsistensi untuk audit.
- Infrastructure as Code & version control — meningkatkan ketertelusuran (setiap perubahan tercatat & dapat diaudit).
- Blameless postmortem & monitoring — memperbaiki sistem tanpa mengubah gerbang regulasi.

Yang tidak dapat diadopsi apa adanya:

- Continuous Deployment otomatis ke produksi — rilis ke perangkat medis memerlukan persetujuan/sertifikasi formal; deployment tak boleh sepenuhnya tanpa gerbang.
- Deployment Frequency tinggi (rilis harian ke produksi) — bertentangan dengan siklus validasi regulator per rilis. (CI/CD boleh berhenti di staging; continuous delivery sampai siap-rilis diperbolehkan, tetapi continuous deployment ke perangkat produksi tidak.)

5.5 Trade-off Kecepatan dan Stabilitas

Berdasarkan pengalaman JOB 2 (manual, lambat, 3 kegagalan) dan JOB 4 (otomatis, cepat, 0 kegagalan), peningkatan Deployment Frequency tidak serta-merta menaikkan Change Failure Rate karena rilis yang sering memaksa batch size kecil: perubahan kecil lebih mudah diuji, ditinjau, dan dipulihkan. Otomasi + health check (seperti pada `setup.sh`) menangkap cacat sebelum ke produksi, sehingga kecepatan justru menurunkan risiko — persis temuan DORA bahwa kedua pilar berkorelasi positif.

Satu kondisi di mana asumsi ini tidak berlaku: bila frekuensi deployment dinaikkan tanpa otomasi pengujian yang memadai — mis. mempercepat rilis sambil melewati/menonaktifkan automated test dan health check. Dalam kondisi ini menaikkan DF akan menaikkan CFR karena cacat lolos ke produksi tanpa jaring pengaman. Kecepatan hanya aman bila ditopang otomasi kualitas & umpan balik cepat (The First & Second Way).

6. Kesimpulan

- Pola serah-terima manual terbukti boros & rawan gagal: Lead Time 14 menit, 3 kegagalan, Flow Efficiency hanya $\pm 45\%$ (JOB 2–3).
- Penerapan The First Way melalui `setup.sh` memangkas Lead Time $\pm 93\%$ menjadi ± 1 menit, menghilangkan kegagalan & kebutuhan komunikasi manual (JOB 4).
- DevOps adalah budaya + proses + pengukuran, bukan sekadar perkakas; kecepatan dan stabilitas berkorelasi positif bila ditopang otomatisasi & umpan balik.
- Seluruh artefak (kode, skrip, VSM, postmortem) tersimpan sebagai artefak kolaborasi di repository, ditambah Challenge "Zero-Touch Verification" yang lolos `shellcheck` bersih.

7. Lampiran

Lampiran A — `app-sentra/src/app.py`

```
from flask import Flask, jsonify
import os
import datetime

app = Flask(__name__)
BUILD = os.getenv("BUILD_ID", "dev-local")

@app.route("/")
def index():
    return jsonify(
        service="sentra-digital-batam",
        status="running",
        build=BUILD,
        time=datetime.datetime.now().isoformat(timespec="seconds"),
    )

@app.route("/health")
def health():
    return jsonify(status="ok"), 200

if __name__ == "__main__":
    # Bind ke 127.0.0.1 (bukan 0.0.0.0) demi keamanan; port dari environment.
    port = int(os.getenv("PORT", "5000"))
    app.run(host="127.0.0.1", port=port)
```

Lampiran B — app-sentra/setup.sh (Zero-Touch)

```
#!/usr/bin/env bash
# setup.sh - Otomasi penyiapan & verifikasi aplikasi (The First Way: Flow)
# ./setup.sh          Setup venv + install + jalankan + smoke test.
# ./setup.sh --check  Validasi prasyarat saja (exit 0=siap, 1=tidak).
set -euo pipefail

APP_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
VENV_DIR="$APP_DIR/.venv"
PORT="${PORT:-5000}"
MAX_PORT_SCAN=20
HEALTH_RETRIES=5
HEALTH_DELAY=2

log() { echo "[setup] $*"; }
fail() { echo "[setup] GAGAL: $*" >&2; exit 1; }

port_in_use() {
    # 0 = terpakai
    local p="$1"
    if command -v ss >/dev/null 2>&1; then
        ss -ltn 2>/dev/null | grep -q ":${p}[:[:space:]]"
    else
        python3 - "$p" <<'PY'
import socket, sys
p = int(sys.argv[1]); s = socket.socket()
try: s.bind(("127.0.0.1", p)); sys.exit(1)
except OSError: sys.exit(0)
finally: s.close()
PY
    fi
}

find_free_port() {
    # cetak port bebas mulai $1
    local start="$1" p end; end=$((start + MAX_PORT_SCAN))
    for ((p = start; p < end; p++)); do
        if ! port_in_use "$p"; then echo "$p"; return 0; fi
    done
    return 1
}

check_prereq() {
    log "[1/5] Memeriksa prasyarat..."
    command -v python3 >/dev/null || fail "python3 tidak ditemukan."
    python3 -c 'import sys; sys.exit(0 if sys.version_info >= (3,10) else 1)' \
        || fail "dibutuhkan Python 3.10 atau lebih baru."
    python3 -m pip --version >/dev/null 2>&1 || python3 -m ensurepip >/dev/null 2>&1 \
        || fail "pip tidak tersedia dan gagal di-bootstrap."
    [ -f "$APP_DIR/requirements.txt" ] || fail "requirements.txt tidak ditemukan."
    log "Prasyarat terpenuhi (python3, pip, requirements.txt)."
}

if [ "${1:-}" = "--check" ]; then
    check_prereq
else
    if port_in_use "$PORT"; then
        log "Port $PORT sedang terpakai."
        if free_port=$(find_free_port "$PORT"); then log "Port bebas: $free_port."
        else fail "tidak ada port bebas pada rentang $PORT-$((PORT + MAX_PORT_SCAN))."; fi
    else log "Port $PORT bebas."; fi
    log "SIAP: seluruh prasyarat lingkungan terpenuhi."; exit 0
fi
```

```

fi

check_prereq
log "[2/5] Menyiapkan virtual environment..."
[ -d "$VENV_DIR" ] || python3 -m venv "$VENV_DIR"
# shellcheck source=/dev/null
source "$VENV_DIR/bin/activate"

log "[3/5] Memasang dependensi terkunci..."
pip install --quiet --upgrade pip
pip install --quiet -r "$APP_DIR/requirements.txt"

if port_in_use "$PORT"; then
    log "Port $PORT terpakai, mencari port bebas berikutnya..."
    PORT="$(find_free_port "$PORT")" || fail "tidak ada port bebas yang tersedia."
    log "Menggunakan port $PORT."
fi

log "[4/5] Menjalankan aplikasi pada port $PORT..."
PORT="$PORT" python3 "$APP_DIR/src/app.py" &
APP_PID=$!
trap 'kill "$APP_PID" 2>/dev/null || true' EXIT

log "[5/5] Melakukan smoke test (retry maks $HEALTH_RETRIES x, jeda ${HEALTH_DELAY}s)..."
ok=0
for ((i = 1; i <= HEALTH_RETRIES; i++)); do
    if curl -fsS "http://127.0.0.1:$PORT/health" >/dev/null 2>&1; then ok=1; break; fi
    log " percobaan $i/$HEALTH_RETRIES belum berhasil, menunggu ${HEALTH_DELAY}s..."
    sleep "$HEALTH_DELAY"
done

if [ "$ok" -eq 1 ]; then
    log "SUKSES: aplikasi berjalan dan lulus health check (PID $APP_PID, port $PORT)."
else
    fail "aplikasi tidak merespons health check setelah $HEALTH_RETRIES percobaan."
fi

log "Aplikasi aktif di http://127.0.0.1:$PORT/ (tekan Ctrl+C untuk berhenti)."
wait "$APP_PID"

```

```
#!/usr/bin/env bash
# teardown.sh - Menghentikan aplikasi & membersihkan .venv secara AMAN.
# ./teardown.sh          Hentikan aplikasi (pertahankan .venv).
# ./teardown.sh --purge  Hentikan + hapus .venv (dengan validasi path).
set -euo pipefail

APP_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
VENV_DIR="$APP_DIR/.venv"
PORT="${PORT:-5000}"
log() { echo "[teardown] $*"; }

stop_app() {
    local pids=""
    if command -v lsof >/dev/null 2>&1; then pids="$(lsof -ti tcp:$PORT 2>/dev/null || true)"
    elif command -v fuser >/dev/null 2>&1; then pids="$(fuser "$PORT"/tcp 2>/dev/null || true)"; fi
    if [ -n "$pids" ]; then
        log "Menghentikan proses pada port $PORT: $pids"
        # shellcheck disable=SC2086
        kill $pids 2>/dev/null || true; sleep 1
    else log "Tidak ada proses aktif pada port $PORT."; fi
}

purge_venv() {
    # hapus HANYA setelah validasi path
    [ -z "${VENV_DIR:-}" ] && { log "Batal: VENV_DIR kosong."; return 0; }
    if [ "$VENV_DIR" = "/" ] || [ "$VENV_DIR" = "$HOME" ]; then
        log "Batal: VENV_DIR menunjuk lokasi berbahaya ($VENV_DIR)."; return 1; fi
    case "$VENV_DIR" in
        "$APP_DIR"/.venv) : ;;
        *) log "Batal: VENV_DIR ($VENV_DIR) di luar direktori aplikasi."; return 1 ;;
    esac
    if [ -d "$VENV_DIR" ]; then
        log "Menghapus virtual environment: $VENV_DIR"; rm -rf -- "$VENV_DIR"
        log "Selesai membersihkan .venv."
    else log "Direktori .venv tidak ditemukan."; fi
}

stop_app
if [ "${1:-}" = "--purge" ]; then purge_venv
else log "Mode aman: .venv dipertahankan (jalankan dengan --purge untuk menghapus)."; fi
log "SELESAI."
```

Lampiran C — `POSTMORTEM.md` (Blameless)

Ringkasan Insiden: Kegagalan Deployment Manual — serah-terima aplikasi sentra-digital-batam gagal dijalankan Ops pada percobaan pertama akibat artefak & instruksi tidak lengkap (manifestasi Wall of Confusion).

Kronologi (relatif T+0 = Ops menerima artefak):

Waktu	Peristiwa
T+0	Ops menerima <code>serah-terima/</code> (<code>src/</code> + <code>HANDOVER.md</code>) tanpa <code>requirements.txt</code> .
T+1'	<code>python3 src/app.py</code> → <code>ModuleNotFoundError: flask</code> .
T+3'	<code>pip install flask global</code> → <code>externally-managed-environment</code> .
T+6'	Versi/pin dependensi tidak diketahui (<code>requirements.txt</code> absen).
T+9'	Ops membuat & mengaktifkan <code>.venv</code> (menebak).
T+14'	Aplikasi merespons <code>curl</code> ; pulih via workaround manual.

Dampak: Lead Time ± 14 menit; 3 kegagalan; ± 4 pertanyaan yang terhalang aturan; sebagian besar waktu = wait/rework.

Akar masalah pada SISTEM (bukan orang): (1) tidak ada kontrak artefak yang dapat dieksekusi mesin; (2) paritas lingkungan Dev/Prod tidak dijamin sistem; (3) kanal komunikasi searah & lambat tanpa umpan balik cepat.

Tindakan perbaikan (peran): otomasi `setup.sh` idempotent (Dev+Ops); pin dependensi (Dev); health check + smoke test (Ops); `.gitignore` sebelum commit (Dev+Ops); standar Python ≥ 3.10 (Ops); repo bersama sebagai sumber kebenaran (Dev+Ops).

Pelajaran: otomasi = dokumentasi yang tak bisa bohong; perkecil batch & percepat umpan balik; perbaiki sistem bukan salahkan orang; paritas lingkungan itu wajib.

Lampiran D — Panduan Git/GitHub & Projects (dijalankan mandiri)

Perintah Git inti (repo private, dosen sebagai collaborator):

```
cd praktikum-devops-m01
git init
git branch -M main
git add .
git status      # PASTIKAN .env & berkas rahasia TIDAK terdaftar
git commit -m "feat: otomasi deployment menggantikan prosedur manual (Minggu 1)"
git remote add origin https://github.com/<username>/praktikum-devops-m01.git
git push -u origin main

# Commit terpisah untuk Challenge (+10):
git commit -m "feat(challenge): zero-touch verification"
```

- Buat repo Private di github.com/new (jangan centang "Add README").
- Saat push, gunakan Personal Access Token (hak minimum, masa berlaku terbatas) — bukan password akun. Jangan simpan token di repo.
- Settings → Collaborators → undang dosen Antoni Haikal (antoni@polibatam.ac.id) & pasangan.
- Projects → New project (Board): buat board Waterfall & DevOps Flow, isi 5 kartu JOB 1–4, jalankan simulasi pemindahan kartu.

Lampiran E — Tautan & Tangkapan Layar (diisi mahasiswa)

Item	Isi
Tautan repository GitHub (private)	https://github.com/_____/praktikum-devops-m01
Collaborator dosen	Antoni Haikal (antoni@polibatam.ac.id) — ditambahkan: ☐

[Tempel tangkapan layar keluaran `./setup.sh` (pesan SUKSES)]

[Tempel tangkapan layar keluaran `shellcheck setup.sh teardown.sh` yang bersih]

[Tempel tangkapan layar kedua board GitHub Projects (Waterfall & DevOps Flow)]

— Selesai —